

1. FOUR PILLARS OF OOP

Pillar: ENCAPSULATION

- Definition: Binding data + methods, hiding internal state
- C++ Implementation: private/protected members + public getters/setters
- Example: Medicine capsule (hide contents inside)

Pillar: ABSTRACTION

- Definition: Hiding implementation, showing only interface
- C++ Implementation: Pure virtual functions (=0), abstract classes
- Example: TV remote (only buttons visible)

Pillar: INHERITANCE

- Definition: Creating new classes from existing ones
- C++ Implementation: class Derived : public Base
- Example: Child inherits from parent

Pillar: POLYMORPHISM

- Definition: Same interface, different behaviors
- C++ Implementation: Virtual functions, function overloading
- Example: "Start" button works differently for car/bike

2. ACCESS SPECIFIERS

private -> Accessible ONLY inside the class protected -> Accessible inside class + derived classes public -> Accessible everywhere

3. INHERITANCE TYPES

class A {}; class B : public A {}; // Public inheritance (most common) class C : protected A {}; // Protected inheritance class D : private A {}; // Private inheritance class E : public A, public B {}; // Multiple inheritance

4. POLYMORPHISM TYPES

COMPILE-TIME (Early Binding):

- Function overloading
- Operator overloading
- Resolved at compile time

RUNTIME (Late Binding):

- Virtual functions
- Function overriding
- Resolved at runtime using vtable

5. VIRTUAL FUNCTIONS

Syntax: class Base { public: virtual void func() = 0; // Pure virtual -> Abstract class virtual ~Base() {} // Virtual destructor (MUST have) };

Rules:

- Base class pointer can point to derived object
- Virtual destructor REQUIRED for proper cleanup
- Constructors cannot be virtual
- Static functions cannot be virtual

6. IMPORTANT DIFFERENCES

ENCAPSULATION vs ABSTRACTION:

- Encapsulation = Data hiding (using private)
- Abstraction = Implementation hiding (using pure virtual)
- Encapsulation is implementation level
- Abstraction is design level

OVERLOADING vs OVERRIDING:

- Overloading = Compile time, same class
- Overriding = Runtime, base + derived classes
- Overriding requires virtual keyword

ABSTRACT CLASS vs CONCRETE CLASS:

- Abstract = Cannot instantiate, has pure virtual function
- Concrete = Can instantiate, no pure virtual functions

7. QUICK CODE SNIPPETS

Class & Object: class Student { public: string name; void display() { cout << name; } }; Student s1;

Diamond Problem Solution: class A { public: void show() { cout << "A"; } }; class B : virtual public A {}; class C : virtual public A {}; class D : public B, public C {}; // Only one copy of A

Virtual Function Example: class Animal { public: virtual void sound() { cout << "Animal sound"; } }; class Dog : public Animal { void sound() override { cout << "Woof"; } }; Animal* ptr = new Dog(); ptr->sound(); // Output: Woof

8. COMMON INTERVIEW QUESTIONS & ANSWERS

Q: Why OOP? A: Code reusability, data security, flexibility, and easier maintenance.

Q: Difference between struct and class? A: Struct members are public by default; class members are private by default.

Q: What is a virtual destructor? A: Ensures derived class destructor is called when deleting through base pointer. Prevents memory leaks.

Q: Can we create object of abstract class? A: No. Cannot instantiate abstract class.

Q: What is size of empty class? A: 1 byte (to ensure different objects have different addresses).

Q: What is the diamond problem? A: Multiple inheritance issue where a class inherits from two classes that inherit from the same base class. Solved using virtual inheritance.

Q: Can constructors be virtual? A: No. Virtual functions require vtable which is set up after constructor runs.

Q: What is a pure virtual function? A: virtual void func() = 0; Makes class abstract, must be overridden.

Q: Difference between override and overload? A: Override = redefining virtual function in derived class Overload = same function name, different parameters

Q: What is vtable and vptr? A: vtable = virtual table (array of function pointers) vptr = virtual pointer (each object points to its class vtable)

9. KEY KEYWORDS

virtual -> Enable runtime polymorphism override -> Explicitly mark overridden function (C++11) final -> Prevent further overriding/inheritance (C++11) explicit -> Prevent implicit conversions mutable -> Allow modification in const functions friend -> Grant access to private members

10. QUICK CHECKLIST BEFORE INTERVIEW

[] Can explain 4 pillars with real-world examples [] Know difference between overloading & overriding [] Understand virtual functions & vtable mechanism [] Know why virtual destructor is needed [] Can explain abstract class vs concrete class [] Understand public/protected/private inheritance [] Know how to solve diamond problem [] Can write simple polymorphic code [] Understand compile-time vs runtime polymorphism [] Know what =0 syntax means for pure virtual

11. MEMORY LAYOUT (Important for Interviews)

Object with Virtual Functions: +-----+ | vptr |-----+ +-----+ +-----+ | Class Virtual Table| data_member | +-----+ + | ... | | ~Class() | +-----+ + | func1() | | func2() | +-----+ +

Size: Data members + 8 bytes (vptr on 64-bit system)

12. SOLID PRINCIPLES (Bonus for Senior Roles)

S - Single Responsibility: One class, one responsibility O - Open-Closed: Open for extension, closed for modification L - Liskov Substitution: Derived must be substitutable for base I - Interface Segregation: Many specific interfaces > one general D - Dependency Inversion: Depend on abstractions, not concretions

=====

END OF NOTES - GOOD LUCK!